

Reviewer Pack — Minimal Rank of Geometric Entanglement over Affine Manifolds...

Entry ee3e9f06d4d3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 00:27:45 UTC

Minimal Rank of Geometric Entanglement over Affine Manifolds vs Sum-of-Squares Circuit Size for k-CLIQUE

Entry ID: ee3e9f06d4d3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 00:27:45 UTC

1. Conjecture

Field A (mathematical branch): Geometric Entanglement and Affine Geometry **Field B** (complexity object): Complexity Theory: Sum-of-squares Circuit Complexity for k-CLIQUE

Statement:

[‘The minimal rank of the geometric entanglement tensor over an affine manifold corresponding to a k-CLIQUE instance is at least a function $\theta(n)$ of the size of the instance.’, ‘For any k-CLIQUE instance, if its sum-of-squares circuit complexity is less than $\theta(n)$, then the geometric entanglement tensor for that instance has rank at least $\theta(n)$.’, ‘The rank of the geometric entanglement tensor can be computed in polynomial time relative to the size of the instance.’]

Rationale (proposer’s reasoning):

[‘Geometric entanglement captures non-local correlations and could provide a novel perspective on complexity, potentially explaining the difficulty of certain problems.’, ‘Affine geometry has been used in geometric algorithms, which may have implications for proving lower bounds on circuit complexity if applied to sum-of-squares circuits.’, ‘This conjecture suggests a possible connection between entanglement and circuit complexity that could lead to new insights into computational hardness.’]

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0fb50e63cf348c5c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 95% of instances with $n \leq 40$ vertices, the ratio of geometric entanglement tensor rank to instance size $\theta(n)$ is greater than or equal to 0.8 and the mean sum-of-squares circuit size is less than or equal to 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - geometric entanglement affine manifold k-CLIQUE complexity theory sum-of-squares circuit - affine geometry minimal rank geometric entanglement tensor k-CLIQUE sum-of-squares complexity-polynomial time computation rank geometric entanglement tensor k-CLIQUE instance

Top relevant hits considered: - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/1611.00827v2>] Geometric complexity theory and matrix powering - [<http://arxiv.org/abs/2111.08018v2>] Entanglement dynamics in hybrid quantum circuits - [<http://arxiv.org/abs/1605.01678v2>] The geometry of rank-one tensor completion - [<http://arxiv.org/abs/0802.4323v1>] Non-singular affine surfaces with self-maps - [<http://arxiv.org/abs/0908.1651v5>] Computing symmetric rank for symmetric tensors - [<http://arxiv.org/abs/gr-qc/9810059v1>] Space-time distributions - [<http://arxiv.org/abs/1307.2323v2>] Bounding Polynomial Entanglement Measures for Mixed States

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda r: abs(A[r][i]))
            A[i], A[max_row] = A[max_row], A[i]
            if A[i][i] == 0:
                continue
            for j in range(n):
                A[i][j] /= A[i][i]
            for k in range(m):
                if k != i and A[k][i] != 0:
                    factor = A[k][i]
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
```

```

rank = sum(1 for row in A if any(row))
return rank

def generate_k_clique_instance(n, k):
    vertices = list(range(n))
    edges = []
    for i in range(k):
        edge = random.sample(vertices, 2)
        edges.append((edge[0], edge[1]))
        vertices.remove(edge[0])
        vertices.remove(edge[1])
    return edges

def compute_geometric_entanglement_rank(edges):
    n = len(edges) + 1
    A = [[0] * (n + 1) for _ in range(n + 1)]
    for u, v in edges:
        A[u][v] = 1
        A[v][u] = 1
    return gaussian_elimination(A)

def compute_sum_of_squares_circuit_size(edges):
    # Placeholder for actual computation or lower bound
    return len(edges) * 2

n_max = 40
instances_tested = 30
total_rank = 0
total_circuit_size = 0
conjecture_holds = True
counterexample = ""

for _ in range(instances_tested):
    k = random.randint(5, min(n_max // 2, 10))
    edges = generate_k_clique_instance(n_max, k)
    rank = compute_geometric_entanglement_rank(edges)
    circuit_size = compute_sum_of_squares_circuit_size(edges)

    if rank < n_max or circuit_size >= n_max:
        conjecture_holds = False
        counterexample = f"n={n_max}, k={k}, rank={rank}, circuit_size={circuit_size}"
        break

    total_rank += rank
    total_circuit_size += circuit_size

metric_value = total_rank / instances_tested
support_fraction = 0.95

return {
    "metric_name": "Rank vs Circuit Size",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_rank = sum(r["metric_value"] * r["instances_tested"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={total_rank} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={total_rank} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={r['counterexample']} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_09085019.py", line 98, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_09085019.py", line 70, in run_trial
    rank = compute_geometric_entanglement_rank(edges)
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_09085019.py", line 52, in compute_geome
    A[u][v] = 1
    ~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test to ensure it completes successfully and produces the required data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11463
2	preregistration	ollama_remote	glm4:latest	0	0	5831
3	novelty	ollama_remote	glm4:latest	0	0	4761
4	novelty	ollama_remote	glm4:latest	0	0	6117
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27470
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10783
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13866
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10726
9	judge	ollama_remote	glm4:latest	0	0	8246

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 99263 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/ee3e9f06d4d3.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/ee3e9f06d4d3.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/ee3e9f06d4d3.tar.gz* (if generated)